

A Appendix

A.1 Hyperspherical Prototype

As a classic approach, prototype-based methods focus on capturing the distance between a given example x and prototypes, where each prototype has an associated class label. x is then assigned to the class of the closest prototype. In the case of Prototypical Networks (Snell, Swersky, and Zemel 2017), the prototype layer generates a distribution over classes for a given example x by applying a softmax function to distances between the prototypes and $f_\phi(x)$ in the embedding space:

$$p_\phi(y = k | x) = \frac{\exp(-d(f_\phi(x), P_k))}{\sum_{k' \in \mathcal{C}} \exp(-d(f_\phi(x), P_{k'}))} \quad , \quad (21)$$

where P_k denotes the prototype of class k ; $d(\cdot)$ denotes distance metric; $\mathcal{C} = \{c_1, c_2, \dots, c_L\}$ denotes the labels. A common approach is to define prototypes as the mean output vector over training examples per class, e.g., $P_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\phi(x_i)$, where (x_i, y_i) denotes training example and its label; S_k denotes the training set. In this paper, we utilize class prototypes that are defined a priori with large margin separation from Hyperspherical Prototype Network (Mettes, Van der Pol, and Snoek 2019).

In order to obtain the predefined hyperspherical prototype with a large margin separation, we aim to distribute the prototypes as uniformly as possible on the hypersphere. In geometry, the problem mentioned above, known as the Tammes problem (Musin and Tarasov 2015), involves the challenge of distributing C points uniformly on a unit hypersphere to maximize the minimum distance between any two points. We adopt the concise optimization method proposed in the Hyperspherical Prototype Networks (Mettes, van der Pol, and Snoek 2019) for determining the data-independent hyperspherical prototypes:

$$\mathcal{L}_{\text{HP}} = \frac{1}{K} \sum_{i=1}^K \max_{j \in \mathcal{C}} (\hat{\mathbf{P}} \hat{\mathbf{P}}^T - 2\mathbf{I})_{ij} \quad , \quad \text{s.t. } \forall \|\hat{\mathbf{P}}\| = 1 \quad . \quad (22)$$

A.2 Details of Datasets

Cora, Citeseer, Pubmed (Planetoid datasets)(Yang, Cohen, and Salakhudinov 2016), and Cora-ML are extensively utilized citation networks. MS-CS(Shchur et al. 2018), based on the Microsoft Academic Graph, is a co-authorship network. The detailed statistics of these datasets are listed in Table 4, while the calculation method for homophily is outlined as follows:

$$h = \frac{1}{|V|} \sum_{i \in V} \left(\frac{\sum_{j \in \mathcal{N}_i} \mathbf{1}(y_i, y_j)}{|\mathcal{N}_i|} \right) \quad , \quad (23)$$

where h is defined as the average ratio of neighborhoods with the same label as the center ratio (Pei et al. 2020). For the standard semi-supervised node classification, we follow the previous works (Kipf and Welling 2017; Chen et al. 2020; Xie, Tam, and Lau 2023), with the 20 labeled nodes per class. For the few-shot semi-supervised node classification, we follow the settings of Meta-PN (Ding et al. 2022).

Dataset	Cora	Citeseer	Pubmed	Cora-ML	MS-CS	ogbn-arxiv
#Nodes	2,708	3,327	19,717	2,995	18,333	169,343
#Edges	5,429	4,732	44,338	8,158	1,166,243	2,315,598
#Features	1,433	3,703	500	2,879	6,805	128
#Classes	7	6	3	7	15	40
Degree	3.9	2.8	4.5	5.5	4.5	7.7
Homophily	0.83	0.72	0.79	0.81	0.83	0.63

Table 4: Brief Statistics of datasets

Moreover, for the ogbn-arxiv dataset, we randomly select 2.5% of the nodes from the training set as labeled data, while maintaining the same validation and test splits as defined in the OGB Benchmark (Hu et al. 2020b). We conducted 10 random splits on ogbn-arxiv, which can be found in the supplementary materials.

A.3 Visualization

In Figure 6 we show the visualization on Planetoid datasets using the representations generated by NormProp from different layers. We employ the t-SNE (Van der Maaten and Hinton 2008) method for dimensionality reduction. According to the Figure 6, we can visually observe the evolution of the node from the unit hypersphere to its final representations.

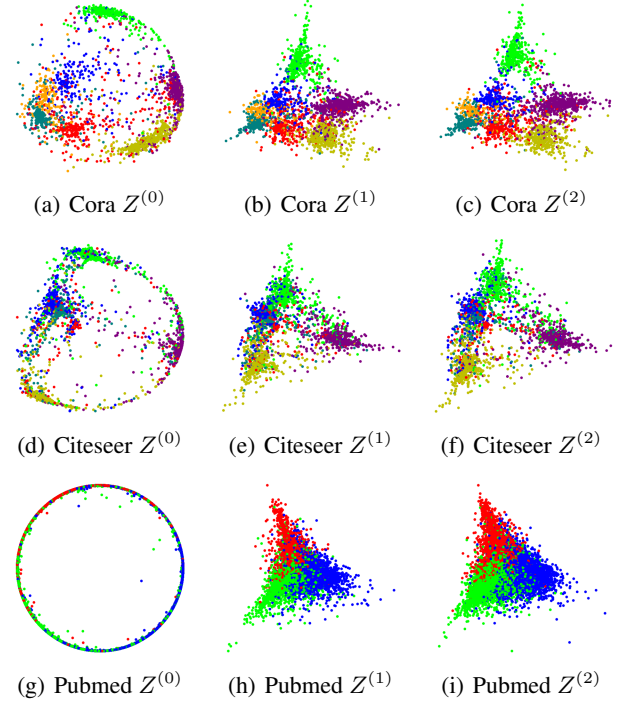


Figure 6: Visualization on Planetoid datasets

A.4 Implementation Details

We implement NormProp in Pytorch (Paszke et al. 2019) and Pytorch Geometric (Fey and Lenssen 2019). We use the two-layer MLP with hidden size in $\{32, 64, 256, 512\}$. We tune the following hyper-parameters: $K \in \{2, 3\}$; $\lambda \in$

$[0, 2]$; weight decay $\in \{1e-2, 5e-3, 1e-3, 5e-4, 1e-4, 0\}$; similarity threshold $\tau \in [0, 1.0]$. Moreover, we set the dimension of hyperspherical space to 8 for standard semi-supervised node classification, 64 for ogbn-arxiv, and 32 for other datasets. We set the dropout rate to 0.3. For ogbn-arxiv, we set the warmup to 100, and for the rest of the datasets, it is 10. The model is trained for 300 epochs, and the best model is selected based on the validation set.

The setup of baseline methods for datasets other than ogbn-arxiv refers to Violin (Xie, Tam, and Lau 2023) and Meta-PN (Ding et al. 2022). For the ogbn-arxiv (Hu et al. 2020b), more details are as follows:

- MLP: We configure the hidden size to 512 for the two-layer MLP.
- LP¹ (Zhu and Ghahramani 2002): We set the α to 0.1, and K to 10.
- GCN² (Kipf and Welling 2017): Following the configuration of OGB benchmark (Hu et al. 2020b), we set the hidden size to 512 and the layers of GCN to 3.
- SGC³ (Wu et al. 2019): We apply the two-layer feature propagation.
- APPNP⁴ (Klicpera, Bojchevski, and Günnemann 2019): Following to the Meta-PN, we set the hidden size to 64, K to 10, and α to 0.1.
- DAGNN⁵ (Liu, Gao, and Ji 2020): We follow the default settings provided by the authors for the best performance.
- C&S⁶ (Huang et al. 2020): We follow the default settings provided by the authors for the best performance.
- Meta-PN⁷ (Ding et al. 2022): We follow the default settings provided by the authors for the best performance.

A.5 Computational Infrastructure Details

All experiments in this paper are conduct on the device given in Table 5. Our code implementation relies on the deep learning framework Pytorch 2.0 (Paszke et al. 2019) and graph neural network library Pytorch Geometric 2.3.1 (Fey and Lenssen 2019).

Device	Value
System	Ubuntu 20.04
GPU	NVIDIA Tesla V100
GPU memory	32G
CUDA version	11.8
CPU	Intel Xeon Gold 6248
Memory	128G

Table 5: Computational infrastructure details.

¹https://github.com/pyg-team/pytorch_geometric/blob/master/examples/label_prop.py

²<https://github.com/snap-stanford/ogb/tree/master/examples/nodeproppred/arxiv>

³<https://github.com/Tiiiger/SGC>

⁴<https://github.com/benedekrozemberczki/APPNP>

⁵<https://github.com/mengliu1998/DeeperGNN>

⁶<https://github.com/CUAI/CorrectAndSmooth>

⁷<https://github.com/kaize0409/Meta-PN>